

# Completing the KMS

14 steps to a REST-fronted key management service

Design impact · proposed solution · test strategy, for every step

**14**

steps in 5 stages

**50**

features short of full coverage

**34**

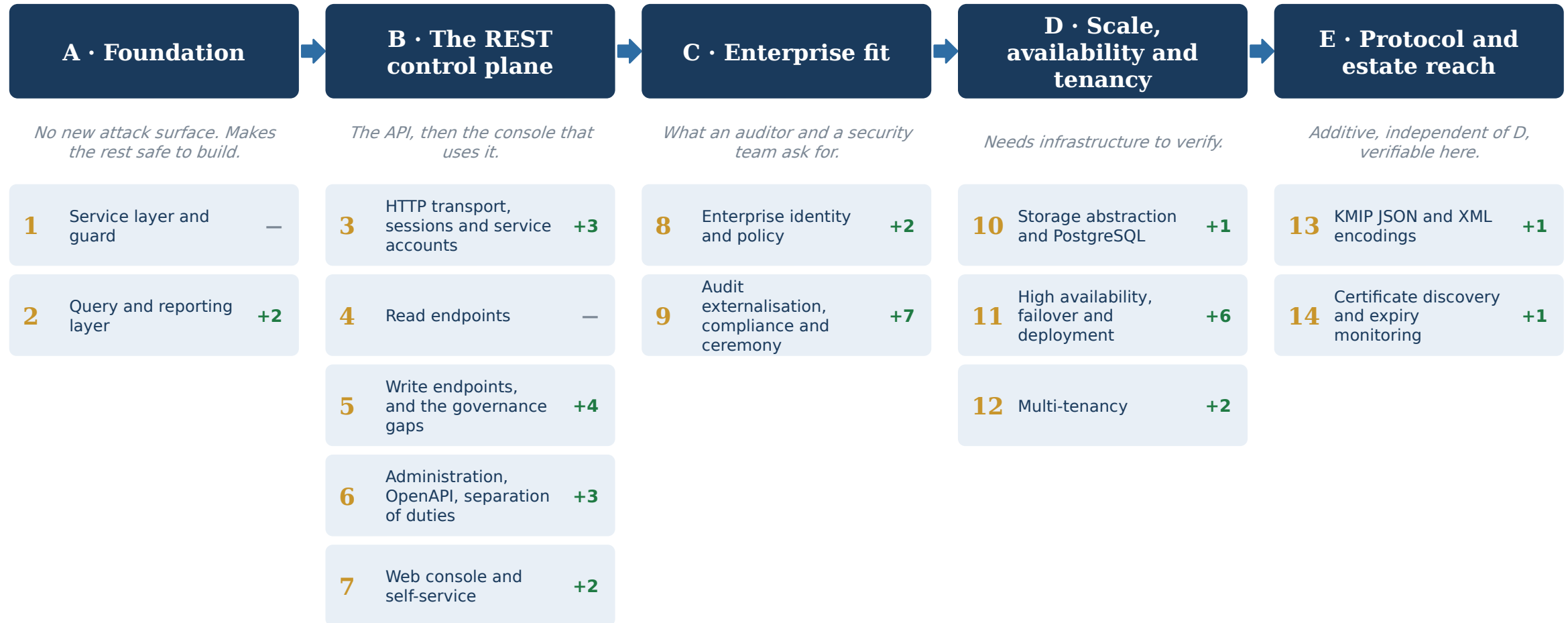
closed by this plan

**16**

deferred, with reasons

## THE PLAN AT A GLANCE

# 5 stages, 14 steps



+n is the number of gap-matrix features the step closes. Steps 1 and 2 close nothing and are the two the rest depends on.

# Service layer and guard

Make authorization, dual control, audit and metrics impossible to bypass — before a second transport exists to bypass them.

Stage A

## WHAT CHANGES IN THE EXISTING DESIGN

- `OperationDispatcher` loses its cross-cutting logic and becomes a codec: decode, call the service, encode.
- `access_control.py` and `dual_control.py` are invoked from one place instead of being wired into a transport.
- No schema change, no configuration change, nothing on the wire changes.

## THE SHAPE OF THE SOLUTION

- A new `kmip_pkcs11/service/` package: `CallContext` (identity, peer address, transport), a `guard()` context manager, and `KeyService` / `AdminService` facades over the existing handlers and store.
- The guard runs the role allowlist, then dual control, then the operation, then audit and metrics — the exact order `dispatch()` uses today.
- `OperationDispatcher` shrinks to: decode TTLV, call the service, encode TTLV.

# Service layer and guard

Make authorization, dual control, audit and metrics impossible to bypass — before a second transport exists to bypass them.

Stage A

## WHAT GETS BUILT

- Move `check_operation_allowed`, `dual_control_enforce`, `_audit` and `_record_metric` out of the dispatcher into the guard. Operation handlers are not touched.
- Every service method takes a `CallContext` as its first argument, so a caller cannot forget to say who is asking.

Closes no gap-matrix feature directly. It exists so that every step after it can be built without re-litigating authorization — and so a later endpoint cannot silently become a path around dual control.

# Service layer and guard

Make authorization, dual control, audit and metrics impossible to bypass — before a second transport exists to bypass them.

Stage A

## WHAT THE TESTS HAVE TO PROVE

- The existing suite must pass unmodified — this step changes no behaviour, and a test that needed editing would mean it did.
- New: a reflection test that enumerates every public method on the service classes and asserts each one is guard-wrapped. This is the test that stops step 5 from quietly reintroducing a bypass.
- New: guard writes an audit row on success and on failure; consumes a dual-control approval before the handler runs, not after; records metrics on both paths.

## GATE

All 816 existing tests pass with no test file modified, and the reflection test proves no service method escapes the guard.

# Query and reporting layer

Let the store answer the questions a console asks, in one query rather than N+1.

Stage A

## WHAT CHANGES IN THE EXISTING DESIGN

- metadata/store.py gains paged query methods; the existing locate() is untouched, so the KMIP handler is unaffected.
- New indexes on the sortable columns — the first schema migration since Phase 1.

## THE SHAPE OF THE SOLUTION

- store.locate\_page(filters, sort, cursor, limit) returning full rows, an opaque cursor and a total — not the bare identifiers Locate returns.
- Cursor is (sort key, uid) encoded, not an offset: offset paging over a table that is being written double-shows rows.
- store.inventory\_report() and store.algorithm\_usage() for the two reporting gaps.

# Query and reporting layer

Let the store answer the questions a console asks, in one query rather than N+1.

Stage A

## WHAT GETS BUILT

- Add the new methods beside the existing locate(), which stays exactly as it is for the KMIP handler.
- Indexes to match the sort columns the console offers.

## CLOSES 2 GAP-MATRIX FEATURES

- Key inventory and discovery
- Crypto-agility reporting

# Query and reporting layer

Let the store answer the questions a console asks, in one query rather than  $N+1$ .

Stage A

## WHAT THE TESTS HAVE TO PROVE

- Paging over a table mutated between pages neither duplicates nor skips a row — insert and delete between fetches and assert the union.
- Sort by each supported column, ascending and descending; total equals a brute-force count; a tampered cursor is rejected rather than trusted.
- Inventory and algorithm counts match a brute-force scan of the same store.

## GATE

A store holding 12,000 objects returns page one of fifty, sorted by expiry, in a single query, with the same figures a full scan produces.



# HTTP transport, sessions and service accounts

An authenticated HTTPS listener that shares nothing with the KMIP port, and cannot be flooded.

Stage B

## WHAT CHANGES IN THE EXISTING DESIGN

- A second listener and a second authenticated entry point: the security surface roughly doubles.
- New config section, new tables for tokens, new failure modes the KMIP path never had (429, CSRF).
- observability.py keeps its own unauthenticated port, deliberately separate.

## THE SHAPE OF THE SOLUTION

- New api: configuration section — own port, TLS mandatory, off by default. Not the observability port: /metrics and /ready are deliberately unauthenticated and must stay that way.
- Opaque 256-bit tokens stored as SHA-256 hashes, with identity, label, created / expires / last used / revoked. Not JWTs: revocation matters more than statelessness, and there is already a database.
- Browsers get an httpOnly, Secure, SameSite cookie plus a CSRF token; machine clients send a bearer header. Long-lived labelled tokens are the service-account mechanism.
- A bounded connection semaphore and a per-identity token bucket.

# HTTP transport, sessions and service accounts

An authenticated HTTPS listener that shares nothing with the KMIP port, and cannot be flooded.

Stage B

## WHAT GETS BUILT

- kmip\_pkcs11/api/ with a small stdlib router on ThreadingHTTPServer — the project has three runtime dependencies and none of them does cryptography, which is a property worth keeping.
- Middleware order: authenticate, rate limit, route, translate exceptions into RFC 9457 problem+json.
- POST and DELETE /api/v1/session, GET /api/v1/me.

## CLOSES 3 GAP-MATRIX FEATURES

- API keys / service accounts
- Connection and rate limiting
- Request bounds and DoS resistance

*Estimated size: ~900 lines, ~35 tests.*

# HTTP transport, sessions and service accounts

An authenticated HTTPS listener that shares nothing with the KMIP port, and cannot be flooded.

Stage B

## WHAT THE TESTS HAVE TO PROVE

- Login succeeds and fails correctly; a disabled identity is refused; a deleted identity's live session stops working immediately.
- Token revocation takes effect on the next request; absolute and idle expiry; a cookie request without a CSRF token is refused while a bearer request is not.
- Rate limiting returns 429 and recovers; the connection cap holds under 200 concurrent connections; the KMIP port is unaffected throughout.
- Configuration validation refuses api.enabled without TLS.
- Every login, logout, token issue and token revoke is an audit row.

## GATE

Two hundred concurrent connections do not exhaust the listener, a revoked token is refused on its next use, and the KMIP port serves normally while the API is under load.

# Read endpoints

Everything a console displays, with authorization identical to KMIP.

Stage B

## WHAT CHANGES IN THE EXISTING DESIGN

- Read paths gain an HTTP representation; the KMIP path does not change at all.
- The 404-versus-403 decision becomes an API contract every later step must preserve.

## THE SHAPE OF THE SOLUTION

- GET /keys (paged, filtered, sorted), /keys/{uid}, /keys/{uid}/attributes, /audit, /audit/verify, /approvals, /reports/expiring, /reports/inventory.
- 404 and 403 are chosen deliberately: an object the caller may not see returns the same answer as one that does not exist, so the API does not become an existence oracle.

# Read endpoints

Everything a console displays, with authorization identical to KMIP.

Stage B

## WHAT GETS BUILT

- Read-only handlers calling the services from step 1 — no store access from the transport layer, which the guard reflection test enforces.

Closes no gap-matrix feature directly. It exists so that every step after it can be built without re-litigating authorization — and so a later endpoint cannot silently become a path around dual control.

# Read endpoints

Everything a console displays, with authorization identical to KMIP.

Stage B

## WHAT THE TESTS HAVE TO PROVE

- Authorization parity: for a given identity, GET /keys returns exactly the set KMIP Locate returns. Parametrised over owner, admin, grantee, group member and stranger.
- The paging contract from step 2 holds through the HTTP layer.
- Audit filters by identity, object, result and time range; the approval queue lists pending requests only.
- A read of an object the caller may not see is indistinguishable from a read of one that does not exist.
- Every read is audited — 'who exported this key' is a read.

## GATE

A read-only console renders key inventory, audit search and the approval queue against real data, and a test proves no write path is reachable through any registered route.

# Write endpoints, and the governance gaps

Lifecycle and delegation over REST, with governance applying by construction rather than by remembering.

Stage B

## WHAT CHANGES IN THE EXISTING DESIGN

- Write paths over HTTP, with governance applied through the guard rather than re-implemented beside it.
- lifecycle/governance.py extends to key pairs, create\_split\_key gains a second method, and the service layer gains transactions.
- The server acquires a data plane. Until now every caller fetched a key reference and asked the token to act on it; now applications send data and never see a key at all.

## THE SHAPE OF THE SOLUTION

- POST /keys; POST /keys/{uid}/activate | revoke | rekey; DELETE /keys/{uid}; PUT /keys/{uid}/cryptoperiod; grants, groups and role permissions; POST /approvals/{id}/approve.
- A dual-control refusal returns 403 carrying the approval request id and the count outstanding, so the console can render 'awaiting 2 approvals' and link to the queue. That falls out of the existing enforcement.
- Three matrix gaps are closed here because they are lifecycle work: asymmetric auto-rotation, Shamir threshold split keys, and batch atomicity via a service-level transaction.
- The data plane beside the control plane: POST /keys/{uid}/encrypt | decrypt | sign | mac, and /datakey returning a fresh key both wrapped under the named key and in the clear — the surface applications actually consume, and the primitives already run inside the token.

# Write endpoints, and the governance gaps

Lifecycle and delegation over REST, with governance applying by construction rather than by remembering.

Stage B

## WHAT GETS BUILT

- All writes go through the guard. Scheduled rotation extends to key pairs using the existing ReKeyKeyPair handler; CreateSplitKey gains a Shamir method alongside XOR; the service layer wraps a batch in one transaction so a mid-batch failure rolls back.
- The crypto endpoints are thin: they translate JSON to the arguments the existing Encrypt, Decrypt, Sign and MAC handlers already take. Re-wrap is decrypt-then-encrypt under the successor key, inside one guarded call so plaintext never leaves the process.

## CLOSES 4 GAP-MATRIX FEATURES

- Automatic rotation
- Split knowledge / M-of-N shares
- Bulk and batch operations
- Encryption as a service (data-plane API)



# Write endpoints, and the governance gaps

Lifecycle and delegation over REST, with governance applying by construction rather than by remembering.

Stage B

## WHAT THE TESTS HAVE TO PROVE

- The dual-control assertion from the KMIP wire test, run verbatim against REST: the first Destroy is refused, the key survives, two other identities approve, the retry succeeds, and the approval is spent.
- Role allowlists and group grants behave identically over both transports — parametrised over the transport, one test body.
- Asymmetric rotation cross-links both new objects to both old ones.
- Shamir: any  $k$  of  $n$  shares reconstruct, any  $k-1$  fail.
- A batch whose third item fails leaves the first two unapplied.
- Ciphertext from the REST encrypt endpoint decrypts over KMIP and the reverse — one test body, both transports, so the two cannot diverge.
- A data key returns a wrapped form that unwraps to the plaintext form; re-wrap to the successor key yields the original plaintext and the old ciphertext no longer decrypts under the new version.

## GATE

The KMIP dual-control test and the REST dual-control test share one body and both pass.

# Administration, OpenAPI, separation of duties

Complete the API surface and split the administrator so no one role can both grant access and use it.

Stage B

## WHAT CHANGES IN THE EXISTING DESIGN

- The admin role splits in two — the first change to the authorization model since Phase 5, and the first needing a migration for existing deployments.
- An OpenAPI document becomes a build artefact CI has to keep in step with the router.

## THE SHAPE OF THE SOLUTION

- Identity, role and group administration endpoints.
- The admin role splits into security-admin (identities, roles, policy) and key-admin (objects, cryptoperiods), so a key administrator cannot grant themselves access to what they administer.
- An OpenAPI 3.1 document generated from the router's route table, not hand-written — a hand-written spec drifts within a month.
- An interactive explorer served from that document, and client libraries generated from it in CI rather than written by hand, for the same reason: anything maintained separately from the router drifts from it.

# Administration, OpenAPI, separation of duties

Complete the API surface and split the administrator so no one role can both grant access and use it.

Stage B

## WHAT GETS BUILT

- Migration assigns both new roles to anyone holding admin today, so an upgrade changes nobody's access until an operator splits them.

## CLOSES 3 GAP-MATRIX FEATURES

- REST / JSON API
- Separation of duties
- OpenAPI specification and generated clients

# Administration, OpenAPI, separation of duties

Complete the API surface and split the administrator so no one role can both grant access and use it.

Stage B

## WHAT THE TESTS HAVE TO PROVE

- A key-admin cannot create an identity; a security-admin cannot read key material; neither can escalate to the other.
- The generated OpenAPI document validates against the 3.1 schema, and a reflection test asserts every registered route appears in it.
- A client generated from the document round-trips a create, read and delete against a live server, so the specification is proved usable and not merely well-formed.
- Responses for a sample of endpoints match their declared schemas.

## GATE

openapi.json validates, covers every route the router knows about, and the two admin roles are provably disjoint in capability.

# Web console and self-service

The interface most of the market considers table stakes, and the reason the API exists.

Stage B

## WHAT CHANGES IN THE EXISTING DESIGN

- A front-end build enters the repository — the first artefact that is neither Python nor documentation tooling.
- Content Security Policy and cookie handling become part of the security review surface.

## THE SHAPE OF THE SOLUTION

- A single-page client of the API. No privileged path: the console can do nothing the API forbids, and holds no credential the API would not accept from curl.
- Views: key inventory, key detail, approval queue, audit search, expiry report, identity and role administration.
- Self-service: a team member creates and rotates keys within their group's grants, without an administrator.

# Web console and self-service

The interface most of the market considers table stakes, and the reason the API exists.

Stage B

## WHAT GETS BUILT

- Served by the API listener or a CDN. Strict CSP with no unsafe-inline. Session in an httpOnly cookie, never in localStorage.

## CLOSES 2 GAP-MATRIX FEATURES

- Web console
- Self-service developer portal

*Estimated size: ~2,000 lines of front-end, ~30 tests.*

# Web console and self-service

The interface most of the market considers table stakes, and the reason the API exists.

Stage B

## WHAT THE TESTS HAVE TO PROVE

- End-to-end with a headless browser (Playwright is available): log in, list keys, open the approval queue, approve a request, watch a blocked Destroy succeed on retry.
- The CSP contains no unsafe-inline and no unsafe-eval.
- A route the API forbids to this identity is not reachable from the console by any means, including direct navigation.

## GATE

An operator completes a full day of routine work — provisioning, approving, investigating an audit question — without touching the CLI.

# Enterprise identity and policy

Authenticate against the corporate directory, and express conditions the role model cannot.

Stage C

## WHAT CHANGES IN THE EXISTING DESIGN

- Identity resolution gains a third path beside password and mTLS certificate.
- The guard gains a policy evaluation point — the first change to its order of checks since step 1.

## THE SHAPE OF THE SOLUTION

- OIDC: validate the ID token (issuer, audience, expiry, signature) and map a claim to a provisioned identity. LDAP/AD bind as an alternative.
- The mTLS path already proves the pattern — an external assertion resolved to a provisioned principal, never inventing one.
- Attribute-based conditions evaluated in the guard before the allowlist: time window, source network, purpose. The engine may only narrow.



# Enterprise identity and policy

Authenticate against the corporate directory, and express conditions the role model cannot.

Stage C

## WHAT GETS BUILT

- Directory group to role mapping, refreshed on login.
- Policies stored in the database, versioned, and every evaluation that denies is audited with the rule that fired.

## CLOSES 2 GAP-MATRIX FEATURES

- Enterprise IdP — LDAP/AD, SAML, OIDC
- Attribute or policy-based access

# Enterprise identity and policy

Authenticate against the corporate directory, and express conditions the role model cannot.

Stage C

## WHAT THE TESTS HAVE TO PROVE

- An expired, wrong-audience, wrong-issuer or unsigned token is refused; a valid token for an unprovisioned subject is refused.
- Directory group changes take effect on next login.
- A policy denies outside its window and permits inside it.
- Property test: for any identity and any policy set, the permitted operation set is a subset of what the role model alone would allow. A policy that widens access is a bug by construction.

## GATE

A corporate login yields exactly the roles the directory says, and a policy denial names the rule that fired in the audit log.

# Audit externalisation, compliance and ceremony

Make the audit trail survive an attacker with file access, and produce the evidence an auditor asks for.

Stage C

## WHAT CHANGES IN THE EXISTING DESIGN

- The audit log stops being purely local: it gains an outbound path and an external dependency.
- Worker 0 takes on a third single-instance responsibility beside the scheduler and the metrics endpoint.
- Startup gains a sealed state. Every health check, every deployment script and every operator runbook has to account for a service that is running but not yet open.

## THE SHAPE OF THE SOLUTION

- A shipper emitting audit rows as syslog/CEF, with a persisted cursor so a restart neither duplicates nor loses entries.
- Periodic notarisation of the chain digest to an append-only external endpoint. This is the one thing the local chain cannot do: detect a wholesale, internally consistent rewrite.
- Compliance report generators over the audit log and key inventory — PCI DSS key-management evidence, NIST SP 800-57 cryptoperiod evidence.
- A key ceremony runbook, and a kmip-admin command producing a signed transcript of what was done, by whom and witnessed by whom — extended to the token PIN itself, split into operator-held shares so the service starts sealed and a quorum opens it. Today one PIN holder is the whole ceremony.
- OpenTelemetry spans around every guarded operation and token call. Metrics say a request was slow; a span says which PKCS#11 call it waited on.
- Alerting rules over the metrics already exported.

# Audit externalisation, compliance and ceremony

Make the audit trail survive an attacker with file access, and produce the evidence an auditor asks for.

Stage C

## WHAT GETS BUILT

- The shipper runs in worker 0 alongside the scheduler, with the same single-instance reasoning.

## CLOSES 7 GAP-MATRIX FEATURES

- Formal key ceremony
- External audit anchoring
- SIEM integration
- Compliance reporting
- Alerting
- Distributed tracing
- Sealed startup with quorum unseal

# Audit externalisation, compliance and ceremony

Make the audit trail survive an attacker with file access, and produce the evidence an auditor asks for.

Stage C

## WHAT THE TESTS HAVE TO PROVE

- The shipper emits every row exactly once across a restart — kill it mid-stream and assert the union at the collector.
- CEF output parses with a standard parser.
- Anchoring detects tampering the local chain cannot: rewrite the whole log consistently, re-chain it so `verify_audit_chain` passes, and assert the anchor comparison fails.
- Report figures match direct store queries.
- A ceremony transcript verifies against its signature and fails after a single byte is changed.
- Any  $k$  of  $n$  unseal shares start the service and any  $k-1$  leave it sealed; a sealed service answers `/health` and refuses every KMIP and REST operation rather than failing obscurely deep in the shim.
- A traced request carries one trace id from the HTTP layer through the guard to the token call, and a sampled-out request costs no spans.

## GATE

A rewritten but internally consistent audit log — one that passes local verification — is caught by the external anchor.

# Storage abstraction and PostgreSQL

Remove the single-writer constraint that blocks everything in stage D.

Stage D

## WHAT CHANGES IN THE EXISTING DESIGN

- metadata/store.py stops being SQLite and becomes an interface with two implementations — the largest structural change in the plan.
- Every SQLite-specific mechanism needs a deliberate equivalent, including the lock that stops the audit chain forking.

## THE SHAPE OF THE SOLUTION

- Extract a Store interface; SQLite becomes one implementation and PostgreSQL another.
- Each SQLite-specific mechanism needs a deliberate equivalent, not a translation: PRAGMA user\_version becomes a schema-version table, json\_extract becomes JSONB operators, RAISE(ABORT) triggers become PL/pgSQL triggers, and BEGIN IMMEDIATE — which is what keeps the audit chain from forking — becomes an advisory lock or SERIALIZABLE.

# Storage abstraction and PostgreSQL

Remove the single-writer constraint that blocks everything in stage D.

Stage D

## WHAT GETS BUILT

- Both backends ship. SQLite stays the default for single-node deployments; it is a good answer there and removing it would be a regression.

## CLOSES 1 GAP-MATRIX FEATURE

- Enterprise database backend

*Estimated size: ~1,500 lines, ~60 tests. Needs a PostgreSQL instance in CI.*

# Storage abstraction and PostgreSQL

Remove the single-writer constraint that blocks everything in stage D.

Stage D

## WHAT THE TESTS HAVE TO PROVE

- The entire existing store suite runs against both backends from one parametrised fixture — not a parallel copy that drifts.
- The audit chain survives concurrent writers on PostgreSQL: hammer it from several connections and assert the chain verifies and no sequence number is reused.
- Migrations apply identically on both; backup and restore work on both.
- A store dump from one backend restores into the other.

## GATE

The full suite — 816 tests plus everything added since — passes against PostgreSQL as well as SQLite.



# High availability, failover and deployment

Survive the loss of a node, a token, or a site.

Stage D

## WHAT CHANGES IN THE EXISTING DESIGN

- Single-process assumptions become cluster-wide ones: the scheduler needs a leader lock, not a worker index.
- pkcs11\_shim gains a pool and health checks — the first change to the shim's contract since the capability probe.

## THE SHAPE OF THE SOLUTION

- Several nodes against one PostgreSQL. The scheduler takes a leader lock so exactly one instance enforces cryptoperiods — the same reasoning that restricts it to worker 0 today, one level up.
- An HSM pool: several tokens, health-checked, with failover. The master key must exist on each, which makes provisioning part of the design rather than an afterthought.
- Scheduled backup with offsite shipping; Helm chart and manifests; the container image and CI workflow finally executed.

# High availability, failover and deployment

Survive the loss of a node, a token, or a site.

Stage D

## WHAT GETS BUILT

- Read traffic spreads across nodes; audited writes still serialise on the chain, so this raises availability more than throughput. Say so rather than implying linear scaling.

## CLOSES 6 GAP-MATRIX FEATURES

- HA clustering
- Multi-site replication and DR
- HSM failover and pooling
- Horizontal throughput
- Scheduled and offsite backup
- Kubernetes / container deployment

*Estimated size: ~1,800 lines, ~55 tests. Needs multiple nodes and a second token.*

# High availability, failover and deployment

Survive the loss of a node, a token, or a site.

Stage D

## WHAT THE TESTS HAVE TO PROVE

- Kill the leader under load: another takes over within the lease period, and no key is deactivated twice.
- Fail a token mid-operation: the request either completes on another or fails cleanly, never silently half-applies.
- Two nodes serving concurrently produce one intact audit chain.
- A scheduled backup taken on one node restores on another.
- The container image builds and its health check passes — the first time this has been executed anywhere.

## GATE

A node is killed under sustained load: no request is lost, the audit chain verifies, and cryptoperiod enforcement continues uninterrupted.

# Multi-tenancy

Let unrelated tenants share one deployment without seeing each other.

Stage D

## WHAT CHANGES IN THE EXISTING DESIGN

- A tenant column reaches every table, every query, the audit log, the CLI and the API.
- Object names stop being globally unique — the change most likely to surprise an existing deployment.

## THE SHAPE OF THE SOLUTION

- Tenant becomes a first-class column on objects, identities and audit rows, and every query is scoped by it.
- Object names become unique per tenant rather than globally — the change most likely to surprise an existing deployment.
- Quotas: object count, operations per second, storage. Per-tenant audit views. Tenant administrators who cannot see other tenants.
- This is why it is last: the boundary reaches into every query, the audit log, the CLI and the API, and half of it would be worse than none.

# Multi-tenancy

Let unrelated tenants share one deployment without seeing each other.

Stage D

## WHAT GETS BUILT

- A migration places every existing object in a default tenant, so a single-tenant deployment behaves exactly as before.

## CLOSES 2 GAP-MATRIX FEATURES

- Multi-tenancy and namespaces
- Per-tenant quotas

*Estimated size: ~1,600 lines, ~80 tests.*

# Multi-tenancy

Let unrelated tenants share one deployment without seeing each other.

Stage D

## WHAT THE TESTS HAVE TO PROVE

- Exhaustive isolation: parametrised over all 41 KMIP operations and every REST route, an identity in tenant B cannot read, modify, locate or destroy an object in tenant A. This is the test that matters and it should be generated from the operation table, not hand-written.
- Name collisions across tenants are allowed; within a tenant they are not.
- Quota enforcement at the boundary, and the audit row that records it.
- A tenant administrator cannot escalate out of their tenant.
- Audit search returns only the caller's tenant.

## GATE

A generated test over every operation and every route proves no cross-tenant reachability exists.

# KMIP JSON and XML encodings

Let a client speak KMIP without implementing a TTLV codec.

Stage E

## WHAT CHANGES IN THE EXISTING DESIGN

- `core/ttlv.py` stops being the only codec, and the tag registry becomes shared infrastructure rather than a TTLV detail.
- Nothing above the codec changes. That is the point of the step: the service layer cannot tell which encoding a request arrived in.

## THE SHAPE OF THE SOLUTION

- The object model does not change. What changes is the wire: a JSON codec and an XML one beside `core/ttlv.py`, both driven by the same tag and type tables, so a new tag is still added in one place.
- An HTTPS binding on the API listener — `POST /kmip`, encoding chosen by `Content-Type` — rather than a second socket. The KMIP TCP port is untouched.
- Encoding is a transport concern, so it sits below the service layer: the same guard, the same handlers, the same audit rows.

# KMIP JSON and XML encodings

Let a client speak KMIP without implementing a TTLV codec.

Stage E

## WHAT GETS BUILT

- core/encodings/ with json.py and xml.py implementing the OASIS profiles, sharing the tag registry the TTLV codec already reads.
- Content negotiation, and 415 for an encoding that is not enabled. Both are off by default: a new parser is new attack surface.

## CLOSES 1 GAP-MATRIX FEATURE

- KMIP JSON and XML encodings



# KMIP JSON and XML encodings

Let a client speak KMIP without implementing a TTLV codec.

Stage E

## WHAT THE TESTS HAVE TO PROVE

- Round-trip every operation's request and response through all three encodings and assert the decoded structures are identical — the existing conformance corpus re-run twice, not a new set of assertions.
- The OASIS JSON and XML test vectors decode to what the TTLV vectors decode to.
- A batch, a wrapped key and a dual-control refusal behave the same over JSON as over TTLV, so an encoding cannot change semantics.
- An unknown tag, a truncated document and a type mismatch are refused with the same error the TTLV codec raises.

## GATE

The conformance suite passes unchanged when re-run against the JSON and XML encodings, and a request refused over TTLV is refused identically over both.

# Certificate discovery and expiry monitoring

Know which certificates the estate is actually running, and warn before one expires.

Stage E

## WHAT CHANGES IN THE EXISTING DESIGN

- The server starts reaching outward. Everything else here waits to be called; this initiates connections, which is a new posture for the service and for its firewall rules.
- The inventory stops being a view of what this KMS holds and becomes a view of the estate, including certificates it does not manage.

## THE SHAPE OF THE SOLUTION

- A scanner that connects to configured TLS endpoints, records the chain presented, and stores subject, issuer, fingerprint and validity window against the inventory.
- Certificates registered here are matched by fingerprint, so the report separates what this KMS manages from what it does not — the second number is the one that matters.
- Expiry thresholds feed the alerting rules built in step 9 rather than opening a second notification path.

# Certificate discovery and expiry monitoring

Know which certificates the estate is actually running, and warn before one expires.

Stage E

## WHAT GETS BUILT

- `kmip_pkcs11/discovery/` with the scanner, a scheduler entry beside the `cryptoperiod` scheduler in worker 0, and the inventory tables.
- Scanning is read-only and rate-limited. This is a tool that touches production endpoints, and it should be incapable of harming them.

## CLOSES 1 GAP-MATRIX FEATURE

- Certificate discovery and expiry monitoring

# Certificate discovery and expiry monitoring

Know which certificates the estate is actually running, and warn before one expires.

Stage E

## WHAT THE TESTS HAVE TO PROVE

- A scan of local TLS servers with known certificates records exactly those chains, intermediate included.
- A certificate registered here and the same certificate found by scanning reconcile to one inventory row, matched on fingerprint.
- An endpoint that times out, presents an expired certificate, or drops the handshake is recorded as such rather than skipped silently.
- An expiry threshold fires once per certificate per window, not on every scan.

## GATE

A scan of a fixture estate reports every certificate, splits managed from unmanaged correctly, and raises an expiry alert exactly once for a certificate inside the warning window.

AFTER STEP 12

# What the plan does not close

INTEGRATION

Cloud BYOK (AWS, Azure, GCP) · Cloud EKM / XKS / hold-your-own-key

SUPPLIER

Algorithm breadth · Post-quantum (ML-KEM, ML-DSA, SLH-DSA) · Key-bound usage policy enforced in hardware

DEMAND

PKCS#11 provider for applications

VALIDATION

Database TDE integration · Storage, backup and VM integrations

OPTIONAL STEP

KMIP 3.0

OUT OF SCOPE

Tokenization / format-preserving encryption

PRODUCT SCOPE

Secrets management · Certificate lifecycle

EXTERNAL

KMIP interoperability certification

PROCUREMENT

FIPS 140-2/3 validated HSM · Common Criteria / eIDAS

*34 of 50 closed by the 14 steps. 8 more are work someone could schedule — integration, vendor validation, or a product decision. The last 8 depend on a supplier, a procurement decision, an OASIS event or hardware this project cannot verify; no amount of planning moves them.*